

The `controller` object keeps a current `view` object, which determines which view is currently displayed. We set it by default to the `BarView` object (the bar chart):

```
self.oView = self.oBarView
```

When the user clicks a radio button, the `controller` sets its current `view` object to the newly selected view and tells the new `view` object to update itself by calling its `update()` method:

```
if self.oBarButton.handleEvent(event):
    self.oView = self.oBarView
    self.oView.update()
elif self.oPieButton.handleEvent(event):
    self.oView = self.oPieView
    self.oView.update()
elif self.oTextButton.handleEvent(event):
    self.oView = self.oTextView
    self.oView.update()
```

At startup, and whenever the user clicks Roll Dice, the controller validates the number of rolls specified in the “Number of rolls” field and tells the model to generate new data:

```
self.oModel.generateRolls(nRounds)
```

All the views are polymorphic, so in each frame the `controller` object calls the `draw()` method of the current `view` object:

```
self.oView.draw() # tell the current view to draw itself
```

The Model

The model is responsible for obtaining (and potentially updating) information. In this program, the `Model` object is simple: it simulates rolling a pair of dice many times, stores the results in instance variables, and reports the data when a `view` object requests it.

When asked to generate data, the model runs a loop simulating rolling the dice and stores its data in two dictionaries: `self.rollsDict`, which uses each